

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Double Linked List”

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

M Galid A



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur penulis ucapkan kehadirat Tuhan Yang Maha Esa karena atas rahmat-Nya laporan praktikum Struktur Data dengan materi Double Linked List ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum mata kuliah Struktur Data. Dalam praktikum ini, penulis mempelajari konsep dasar Double Linked List serta implementasi operasi dasar seperti menambah, menghapus, dan menampilkan data menggunakan bahasa pemrograman Java.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik.

Penulis juga mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, dan teman-teman yang telah membantu selama proses praktikum.

Akhir kata, semoga laporan ini dapat bermanfaat bagi pembaca dan menambah pemahaman mengenai Double Linked List.

Wassalamu'alaikum Warahmatullahi Wabarakatuh

Padang, 13 Mei 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Pengertian Double Linked List	3
2.2 Implementasi Double Linked List pada Praktikum.....	3
2.2.1 Kode Pertama(NodeDLL_2511531013)	3
2.2.2 Kode Kedua(insertDLL_2511531013)	4
2.2.3 Kode Ketiga(PenelusuranDLL_2511531013).....	6
2.2.4 Kode Keempat(HapusDLL_2511531013)	8
BAB III PENUTUP	11
3.1 Kesimpulan	11
3.2 Saran.....	11
DAFTAR PUSTAKA	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data yang terstruktur sangat penting agar proses penyimpanan dan pengolahan data dapat dilakukan dengan lebih efisien. Salah satu konsep dasar dalam struktur data yang digunakan untuk mengelola data secara dinamis adalah Linked List. Setelah mempelajari Single Linked List, pengembangan selanjutnya adalah Double Linked List yang memiliki kemampuan lebih fleksibel dalam proses penelusuran data.

Double Linked List merupakan struktur data linear yang setiap node-nya memiliki dua pointer, yaitu pointer menuju node berikutnya (next) dan pointer menuju node sebelumnya (prev). Dengan adanya dua arah hubungan ini, proses traversal data dapat dilakukan dari depan ke belakang maupun dari belakang ke depan.

Pada praktikum ini, dilakukan implementasi Double Linked List menggunakan bahasa pemrograman Java. Praktikum berfokus pada operasi dasar seperti menambah node di awal, akhir, dan posisi tertentu, menghapus node, serta melakukan penelusuran maju dan mundur pada linked list.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara kerja Double Linked List serta penerapannya dalam pengelolaan data yang lebih fleksibel dibandingkan Single Linked List.

1.2 Tujuan

Adapun tujuan dari praktikum Struktur Data dengan materi Double Linked List ini adalah sebagai berikut:

1. Mahasiswa mampu memahami konsep dasar Double Linked List.

2. Mahasiswa mampu memahami fungsi pointer next dan prev pada setiap node.
3. Mahasiswa mampu mengimplementasikan operasi penambahan node pada Double Linked List.
4. Mahasiswa mampu melakukan penghapusan node pada posisi tertentu dalam Double Linked List.
5. Mahasiswa mampu melakukan traversal maju dan mundur pada Double Linked List menggunakan bahasa pemrograman Java.

1.3 Manfaat

Adapun manfaat dari praktikum Struktur Data dengan materi Double Linked List ini adalah sebagai berikut:

1. Mahasiswa dapat memahami cara kerja struktur data dinamis menggunakan dua arah pointer.
2. Mahasiswa dapat meningkatkan kemampuan logika pemrograman dalam pengelolaan data menggunakan linked list.
3. Mahasiswa dapat memahami proses traversal data secara maju maupun mundur pada Double Linked List.
4. Mahasiswa dapat melatih kemampuan implementasi struktur data menggunakan bahasa pemrograman Java.
5. Mahasiswa dapat menjadikan materi Double Linked List sebagai dasar dalam mempelajari struktur data lain yang lebih kompleks.

BAB II PEMBAHASAN

2.1 Pengertian Double Linked List

Double Linked List adalah jenis daftar berantai di mana node berisi informasi dan dua pointer, yaitu pointer kiri dan pointer kanan. Pointer kiri pada daftar berantai ganda menunjuk ke node sebelumnya dan pointer kanan menunjuk ke node berikutnya dalam daftar berantai. Node pertama dari daftar berantai ganda memiliki nilai NULL pada pointer kirinya dan node terakhir dari daftar berantai ganda memiliki nilai NULL pada pointer kanannya. Daftar ini juga dikenal sebagai daftar berantai dua arah karena memiliki dua pointer. Keuntungan dari daftar berantai ganda ini adalah kita sekarang dapat melakukan navigasi ke kedua arah. Selain itu, kita dapat menghapus sebuah node tanpa menyertakan alamat node sebelumnya karena setiap node memiliki pointer kiri yang menunjuk ke node sebelumnya.

2.2 Implementasi Double Linked List pada Praktikum

2.2.1 Kode Pertama(NodeDLL_2511531013)

```
1 package Pekan6_2511531013;
2
3 public class NodeDLL_2511531013 {
4     //Mendefenisikan kelas Node
5     int data_1013;
6     NodeDLL_2511531013 next_1013;
7     NodeDLL_2511531013 prev_1013;
8
9     //konstruktor
10    public NodeDLL_2511531013(int data_1013) {
11        this.data_1013=data_1013;
12        this.next_1013=null;
13        this.prev_1013=null;    }
14    }
15 }
16
```

Kode Program 2.1

Program NodeDLL_2511531013 digunakan untuk membuat simpul atau node pada Double Linked List. Di dalam class ini

terdapat tiga atribut utama, yaitu data_1013, next_1013, dan prev_1013.

Atribut data_1013 berfungsi untuk menyimpan nilai data pada node. Atribut next_1013 digunakan untuk menunjuk ke node berikutnya, sedangkan prev_1013 digunakan untuk menunjuk ke node sebelumnya.

Pada constructor NodeDLL_2511531013(int data_1013), nilai data akan dimasukkan ke dalam node baru. Kemudian pointer next_1013 dan prev_1013 diatur bernilai null karena saat node pertama kali dibuat, node tersebut belum terhubung dengan node lain.

2.2.2 Kode Kedua(insertDLL_2511531013)

```
1 package Pekan6_2511531013;
2
3 public class InsertDLL_2511531013 {
4     // menambah node di awal DLL
5     static NodeDLL_2511531013 insertBegin(NodeDLL_2511531013 head_1013, int data_1013) {
6         // buat node baru
7         NodeDLL_2511531013 new_node_1013 = new NodeDLL_2511531013(data_1013);
8         // jadikan pointer nextnya head
9         new_node_1013.next_1013 = head_1013;
10        // jadikan pointer prev head ke new_node
11        if (head_1013 != null) {
12            head_1013.prev_1013 = new_node_1013;
13        }
14        return new_node_1013;
15    }
16    // fungsi menambahkan node di akhir
17    public static NodeDLL_2511531013 insertEnd(NodeDLL_2511531013 head_1013, int new_data_1013) {
18        // buat node baru
19        NodeDLL_2511531013 new_node_1013 = new NodeDLL_2511531013(new_data_1013);
20        // jika dll null jadikan head
21        if (head_1013 == null) {
22            head_1013 = new_node_1013;
23        }
24        else {
25            NodeDLL_2511531013 curr_1013 = head_1013;
26            while (curr_1013.next_1013 != null) {
27                curr_1013 = curr_1013.next_1013;
28            }
29            curr_1013.next_1013 = new_node_1013;
30            new_node_1013.prev_1013 = curr_1013;
31        }
32        return head_1013;
33    }
34 }
35 //fungsi menambahkan node di posisi tertentu
36 public static NodeDLL_2511531013 insertAtPosition(NodeDLL_2511531013 head_1013, int pos_1013, int new_data_1013) {
37     // buat node baru
38     NodeDLL_2511531013 new_node_1013 = new NodeDLL_2511531013(new_data_1013);
39     if (pos_1013 == 1) {
40         new_node_1013.next_1013 = head_1013;
41         if (head_1013 != null) {
42             head_1013.prev_1013 = new_node_1013;
43         }
44         head_1013 = new_node_1013;
45         return head_1013;
46     }
47     NodeDLL_2511531013 curr_1013 = head_1013;
48     for (int i = 1; i < pos_1013 - 1 && curr_1013 != null; ++i) {
49         curr_1013 = curr_1013.next_1013;
50     }
51     if (curr_1013 == null) {
52         System.out.println("Posisi tidak ada");
53         return head_1013;
54     }
55     new_node_1013.prev_1013 = curr_1013;
56     new_node_1013.next_1013 = curr_1013.next_1013;
57     curr_1013.next_1013 = new_node_1013;
58     if (new_node_1013.next_1013 != null) {
59         new_node_1013.next_1013.prev_1013 = new_node_1013;
60     }
61     return head_1013;
62 }
```

```

63 public static void printList(NodeDLL_2511531013 head_1013) {
64     NodeDLL_2511531013 curr_1013 = head_1013;
65     while (curr_1013 != null) {
66         System.out.print(curr_1013.data_1013 + " <-> ");
67         curr_1013 = curr_1013.next_1013;
68     }
69     System.out.println();
70 }

71 public static void main(String[] args) {
72     // membuat dll 2 <-> 3 <-> 5
73     NodeDLL_2511531013 head_1013 = new NodeDLL_2511531013(2);
74     head_1013.next_1013 = new NodeDLL_2511531013(3);
75     head_1013.next_1013.prev_1013 = head_1013;
76     head_1013.next_1013.next_1013 = new NodeDLL_2511531013(5);
77     head_1013.next_1013.next_1013.prev_1013 = head_1013.next_1013;
78     // cetak DLL awal
79     System.out.print("DLL Awal: ");
80     printList(head_1013);
81     // tambah 1 di awal
82     head_1013 = insertBegin(head_1013, 1);
83     System.out.print(
84         "simpul 1 ditambah di awal: ");
85     printList(head_1013);
86     // tambah 6 di akhir
87     System.out.print(
88         "simpul 6 ditambah di akhir: ");
89     int data_1013 = 6;
90     head_1013 = insertEnd(head_1013, data_1013);
91     printList(head_1013);
92     // menambah node 4 di posisi 4
93     System.out.print("tambah node 4 di posisi 4: ");
94     int data2_1013 = 4;
95     int pos_1013 = 4;
96     head_1013 = insertAtPosition(head_1013, pos_1013, data2_1013);
97     printList(head_1013);
98 }
99 }

```

Kode Program 2.2

Program `insertDLL_2511531013` digunakan untuk melakukan proses penambahan node pada Double Linked List. Pada program ini terdapat beberapa method yang digunakan untuk menambahkan node di awal, di akhir, dan pada posisi tertentu.

Method `insertBegin()` digunakan untuk menambahkan node di bagian awal Double Linked List. Program akan membuat node baru, kemudian pointer `next_1013` dari node baru diarahkan ke head lama. Setelah itu, pointer `prev_1013` dari head lama dihubungkan ke node baru sehingga node baru menjadi head baru.

Method `insertEnd()` digunakan untuk menambahkan node di bagian akhir Double Linked List. Program terlebih dahulu membuat node baru, kemudian melakukan penelusuran sampai ke node terakhir. Setelah node terakhir ditemukan, pointer `next_1013` dari

node terakhir diarahkan ke node baru dan pointer prev_1013 dari node baru diarahkan ke node terakhir.

Selanjutnya, method insertAtPosition() digunakan untuk menambahkan node pada posisi tertentu. Program akan melakukan penelusuran sampai posisi yang diinginkan ditemukan. Setelah posisi ditemukan, pointer next_1013 dan prev_1013 akan diperbarui agar node baru dapat disisipkan di antara node yang sudah ada.

Program juga memiliki method printList() yang digunakan untuk menampilkan seluruh isi Double Linked List dari awal sampai akhir.

Pada method main(), program membuat Double Linked List awal dengan data 2 <-> 3 <-> 5. Setelah itu dilakukan beberapa proses penambahan node, yaitu menambahkan node 1 di awal, node 6 di akhir, dan node 4 pada posisi ke-4.

2.2.3 Kode Ketiga(PenelusuranDLL_2511531013)

```
1 package Pekan6_2511531013;
2 public class PenelusuranDLL_2511531013 {
3     //fungsi penelusuran maju
4     static void forwardTraversal(NodeDLL_2511531013 head_1013) {
5         // memulai penelusuran dari head
6         NodeDLL_2511531013 curr_1013 = head_1013;
7         //lanjutkan sampai akhir
8         while (curr_1013 != null) {
9             //print data
10            System.out.print(curr_1013.data_1013 + " <-> ");
11            //pindah ke node berikutnya
12            curr_1013 = curr_1013.next_1013;
13        }
14        //print spasi
15        System.out.println();
16    }
17    // fungsi penelusuran mundur
18    static void backwardTraversal(NodeDLL_2511531013 tail_1013) {
19        // mulai dari akhir
20        NodeDLL_2511531013 curr_1013 = tail_1013;
21        //lanjut sampai head
22        while (curr_1013 != null) {
23            //cetak data
24            System.out.print(curr_1013.data_1013 + " <-> ");
25            // pindah ke node sebelumnya
26            curr_1013 = curr_1013.prev_1013;
27        }
28        // cetak spasi
29        System.out.println();
30    }
31 }
```

```

31 public static void main(String[] args) {
32     // cetak DLL
33     NodeDLL_2511531013 head_1013 = new NodeDLL_2511531013(1);
34     NodeDLL_2511531013 second_1013 = new NodeDLL_2511531013(2);
35     NodeDLL_2511531013 third_1013 = new NodeDLL_2511531013(3);
36
37     head_1013.next_1013 = second_1013;
38     second_1013.prev_1013 = head_1013;
39     second_1013.next_1013 = third_1013;
40     third_1013.prev_1013 = second_1013;
41
42     System.out.println("Penelusuran maju:");
43     forwardTraversal(head_1013);
44
45     System.out.println("Penelusuran mundur:");
46     backwardTraversal(third_1013);
47 }
48 }

```

Kode Program 2.3

Program PenelusuranDLL_2511531013 digunakan untuk melakukan proses penelusuran data pada Double Linked List. Pada program ini, penelusuran dilakukan dalam dua arah, yaitu penelusuran maju (forward traversal) dan penelusuran mundur (backward traversal).

Method forwardTraversal() digunakan untuk melakukan penelusuran dari node pertama menuju node terakhir. Program memulai penelusuran dari head, kemudian menampilkan data pada setiap node satu per satu menggunakan pointer next_1013 sampai mencapai node terakhir.

Method backwardTraversal() digunakan untuk melakukan penelusuran dari node terakhir menuju node pertama. Program memulai penelusuran dari tail atau node terakhir, kemudian menampilkan data pada setiap node menggunakan pointer prev_1013 sampai mencapai node pertama.

Pada method main(), program membuat tiga node dengan data 1, 2, dan 3. Setelah itu, masing-masing node dihubungkan menggunakan pointer next_1013 dan prev_1013 sehingga membentuk Double Linked List.

Setelah proses penghubungan selesai, program melakukan penelusuran maju menggunakan method `forwardTraversal()` dan penelusuran mundur menggunakan method `backwardTraversal()`.

2.2.4 Kode Keempat(HapusDLL_2511531013)

```

1 package Pekas_2511531013;
2 public class HapusDLL_2511531013 {
3     // fungsi menghapus node awal
4     public static NodeDLL_2511531013 delHead(NodeDLL_2511531013 head_1013) {
5         if (head_1013 == null) {
6             return null;
7         }
8         NodeDLL_2511531013 temp_1013 = head_1013;
9         head_1013 = head_1013.next_1013;
10        if (head_1013 != null) {
11            head_1013.prev_1013 = null;
12        }
13        return head_1013;
14    }
15    // fungsi menghapus di akhir
16    public static NodeDLL_2511531013 delLast(NodeDLL_2511531013 head_1013) {
17        if (head_1013 == null) {
18            return null;
19        }
20        if (head_1013.next_1013 == null) {
21            return null;
22        }
23        NodeDLL_2511531013 curr_1013 = head_1013;
24        while (curr_1013.next_1013 != null) {
25            curr_1013 = curr_1013.next_1013;
26        }
27        // update pointer previous node
28        if (curr_1013.prev_1013 != null) {
29            curr_1013.prev_1013.next_1013 = null;
30        }
31        return head_1013;
32    }
33    // fungsi menghapus node posisi tertentu
34    public static NodeDLL_2511531013 delPos(NodeDLL_2511531013 head_1013, int pos_1013) {
35        // jika DLL kosong
36        if (head_1013 == null) {
37            return head_1013;
38        }
39        NodeDLL_2511531013 curr_1013 = head_1013;
40        // telusuri sampai ke node yang akan dihapus
41        for (int i = 1; curr_1013 != null && i < pos_1013; ++i) {
42            curr_1013 = curr_1013.next_1013;
43        }
44        // jika posisi tidak ditemukan
45        if (curr_1013 == null) {
46            return head_1013;
47        }
48        // Update pointer
49        if (curr_1013.prev_1013 != null) {
50            curr_1013.prev_1013.next_1013 = curr_1013.next_1013;
51        }
52        if (curr_1013.next_1013 != null) {
53            curr_1013.next_1013.prev_1013 = curr_1013.prev_1013;
54        }
55        // jika yang dihapus head
56        if (head_1013 == curr_1013) {
57            head_1013 = curr_1013.next_1013;
58        }
59        return head_1013;
60    }
61    // fungsi mencetak DLL
62    public static void printList(NodeDLL_2511531013 head_1013) {
63        NodeDLL_2511531013 curr_1013 = head_1013;
64        while (curr_1013 != null) {
65            System.out.print(curr_1013.data_1013 + " <-> ");
66            curr_1013 = curr_1013.next_1013;
67        }
68        System.out.println();
69    }
70 }

```

```

62 public static void main(String[] args) {
63     // buat sebuah DLL
64     NodeDLL_2511531013 head_1013 = new NodeDLL_2511531013(1);
65     head_1013.next_1013 = new NodeDLL_2511531013(2);
66     head_1013.next_1013.prev_1013 = head_1013;
67     head_1013.next_1013.next_1013 = new NodeDLL_2511531013(3);
68     head_1013.next_1013.next_1013.prev_1013 = head_1013.next_1013;
69     head_1013.next_1013.next_1013.next_1013 = new NodeDLL_2511531013(4);
70     head_1013.next_1013.next_1013.next_1013.prev_1013 = head_1013.next_1013.next_1013;
71     head_1013.next_1013.next_1013.next_1013.next_1013 = new NodeDLL_2511531013(5);
72     head_1013.next_1013.next_1013.next_1013.next_1013.prev_1013 = head_1013.next_1013.next_1013.next_1013;
73
74     System.out.print("DLL Awal: ");
75     printList(head_1013);
76
77     System.out.print("Setelah head dihapus: ");
78     head_1013 = delHead(head_1013);
79     printList(head_1013);
80
81     System.out.print("Setelah node terakhir dihapus: ");
82     head_1013 = delLast(head_1013);
83     printList(head_1013);
84
85     System.out.print("menghapus node ke 2: ");
86     head_1013 = delPos(head_1013, 2);
87
88     printList(head_1013);
89 }
90 }

```

Kode Program 2.4

Program HapusDLL_2511531013 digunakan untuk melakukan proses penghapusan node pada Double Linked List. Pada program ini terdapat beberapa method yang digunakan untuk menghapus node di awal, di akhir, dan pada posisi tertentu.

Method delHead() digunakan untuk menghapus node pertama atau head pada Double Linked List. Program akan memindahkan head ke node berikutnya, kemudian pointer prev_1013 dari head baru diubah menjadi null agar tidak lagi terhubung dengan node sebelumnya.

Method delLast() digunakan untuk menghapus node terakhir pada Double Linked List. Program akan melakukan penelusuran sampai menemukan node terakhir. Setelah node terakhir ditemukan, pointer next_1013 dari node sebelumnya diubah menjadi null sehingga node terakhir terhapus dari list.

Selanjutnya, method delPos() digunakan untuk menghapus node pada posisi tertentu. Program akan melakukan penelusuran sampai posisi yang ingin dihapus ditemukan. Setelah node ditemukan, pointer dari node sebelum dan sesudahnya diperbarui agar node tersebut tidak lagi terhubung dalam Double Linked List.

Program juga memiliki method `printList()` yang digunakan untuk menampilkan isi Double Linked List setelah proses penghapusan dilakukan.

Pada method `main()`, program membuat Double Linked List awal dengan data 1 <-> 2 <-> 3 <-> 4 <-> 5. Setelah itu dilakukan beberapa proses penghapusan, yaitu menghapus node pertama, menghapus node terakhir, dan menghapus node pada posisi ke-2.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Struktur Data dengan materi Double Linked List yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan struktur data linear dinamis yang setiap node-nya memiliki dua pointer, yaitu pointer menuju node berikutnya (next) dan pointer menuju node sebelumnya (prev).

Pada praktikum ini, mahasiswa berhasil mengimplementasikan beberapa operasi dasar pada Double Linked List menggunakan bahasa pemrograman Java, seperti penambahan node di awal, akhir, dan posisi tertentu, penghapusan node, serta penelusuran data secara maju dan mundur.

Selain itu, praktikum ini juga menunjukkan bahwa penggunaan dua pointer pada Double Linked List membuat proses traversal data menjadi lebih fleksibel dibandingkan Single Linked List karena data dapat diakses dari dua arah.

3.2 Saran

Adapun saran yang dapat diberikan setelah pelaksanaan praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan lebih sering berlatih membuat program Double Linked List agar lebih memahami cara kerja pointer dan hubungan antar node.
2. Dalam pembuatan program, sebaiknya dilakukan pengecekan kondisi seperti linked list kosong atau posisi data yang tidak tersedia agar program dapat berjalan lebih aman.
3. Praktikum selanjutnya sebaiknya diberikan contoh kasus yang lebih kompleks agar mahasiswa dapat memahami penerapan Double Linked List dalam permasalahan nyata.
4. Mahasiswa disarankan untuk mempelajari struktur data lain yang berkaitan, seperti Circular Linked List dan Queue, untuk menambah pemahaman mengenai struktur data dinamis.

DAFTAR PUSTAKA

- [1] GeeksforGeeks. “Applications, Advantages and Disadvantages of Doubly Linked List.” Tersedia pada: <https://www.geeksforgeeks.org/dsa/applications-advantages-and-disadvantages-of-doubly-linked-list/> . Diakses pada 13 Mei 2026.

**LAPORAN TUGAS PEKAN 6
STRUKTUR DATA**

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

1. Lagu_2511531013

```
1 package Pekan6_2511531013;
2
3 public class Lagu_2511531013 {
4
5     // atribut lagu
6     private String judul_1013;
7     private String penyanyi_1013;
8
9     // pointer next dan prev
10    Lagu_2511531013 next_1013;
11    Lagu_2511531013 prev_1013;
12
13    // konstruktor
14    public Lagu_2511531013(String judul_1013,
15        String penyanyi_1013) {
16
17        this.judul_1013 = judul_1013;
18        this.penyanyi_1013 = penyanyi_1013;
19
20        // awal node belum terhubung
21        this.next_1013 = null;
22        this.prev_1013 = null;
23    }
24
25    // getter judul
26    public String getJudul_1013() {
27        return judul_1013;
28    }
29
30    // getter penyanyi
31    public String getPenyanyi_1013() {
32        return penyanyi_1013;
33    }
34
35    // setter judul
36    public void setJudul_1013(String judul_1013) {
37        this.judul_1013 = judul_1013;
38    }
39
40    // setter penyanyi
41    public void setPenyanyi_1013(String penyanyi_1013) {
42        this.penyanyi_1013 = penyanyi_1013;
43    }
44
45    // menampilkan data lagu
46    @Override
47    public String toString() {
48        return "Judul : " + judul_1013 +
49            ", Penyanyi : " + penyanyi_1013;
50    }
51 }
```

Program Lagu_2511531013 digunakan untuk membuat struktur dasar node pada Double Linked List playlist lagu. Dalam Double Linked List, setiap node memiliki data serta dua pointer yang digunakan untuk menghubungkan node sebelumnya dan node berikutnya.

Pada program ini terdapat variabel judul_1013 yang digunakan untuk menyimpan judul lagu dan variabel penyanyi_1013 yang digunakan untuk menyimpan nama penyanyi. Selain itu, terdapat pointer next_1013 yang berfungsi menunjuk lagu berikutnya dan pointer prev_1013 yang berfungsi menunjuk lagu sebelumnya.

Program juga memiliki constructor Lagu_2511531013(String judul_1013, String penyanyi_1013) yang digunakan untuk membuat node lagu baru. Constructor ini menerima parameter berupa judul lagu dan nama penyanyi yang kemudian disimpan ke dalam atribut objek.

Saat node lagu pertama kali dibuat, pointer next_1013 dan prev_1013 diatur bernilai null karena node tersebut belum terhubung dengan node lain dalam playlist.

Method getter digunakan untuk mengambil data judul dan penyanyi, sedangkan method setter digunakan untuk mengubah data lagu. Selain itu, method toString() digunakan agar data lagu dapat langsung ditampilkan dengan format yang lebih rapi.

2. Musik_2511531013

```
1 package Pekano_2511532029;  
2  
3 import java.util.Scanner;  
4  
5 public class Musik_2511531013 {  
6  
7     // head double linked list  
8     Lagu_2511531013 head_1013;  
9  
10    // tambah lagu di akhir playlist  
11    public void tambahLagu_1013(String judul_1013,  
12                                String penyanyi_1013) {  
13  
14        // membuat node lagu baru  
15        Lagu_2511531013 laguBaru_1013 =  
16            new Lagu_2511531013(  
17                judul_1013,  
18                penyanyi_1013);  
19  
20        // jika playlist kosong  
21        if (head_1013 == null) {  
22            head_1013 = laguBaru_1013;  
23        } else {  
24  
25            // traversal ke node terakhir  
26            Lagu_2511531013 curr_1013 = head_1013;  
27            while (curr_1013.next_1013 != null) {  
28                curr_1013 = curr_1013.next_1013;  
29            }  
30  
31            // menghubungkan node terakhir ke node baru  
32            curr_1013.next_1013 = laguBaru_1013;  
33  
34            // menghubungkan prev node baru  
35            laguBaru_1013.prev_1013 = curr_1013;  
36        }  
37        System.out.println("Lagu berhasil ditambahkan!");  
38    }  
39  
40    // menghapus lagu pertama  
41    public void hapusLaguAwal_1013() {  
42  
43        // cek playlist kosong  
44        if (head_1013 == null) {  
45            System.out.println("Playlist kosong.");  
46            return;  
47        }  
48  
49        System.out.println(  
50            "Lagu dihapus : "  
51            + head_1013);  
52  
53        // head digeser ke node berikutnya  
54        head_1013 = head_1013.next_1013;  
55  
56        // prev head baru dibuat null  
57        if (head_1013 != null) {  
58            head_1013.prev_1013 = null;  
59        }  
60    }  
61  
62    --  
63  
64
```

```

65 }
66
67 // menampilkan playlist dari awal ke akhir
68 public void tampilMaju_1013() {
69
70     // cek playlist kosong
71     if (head_1013 == null) {
72
73         System.out.println("Playlist kosong.");
74         return;
75     }
76
77     Lagu_2511531013 curr_1013 = head_1013;
78
79     System.out.println("Playlist Maju:");
80
81     // traversal maju
82     while (curr_1013 != null) {
83
84         System.out.println(curr_1013);
85
86         curr_1013 = curr_1013.next_1013;
87     }
88 }
89
90 // menampilkan playlist dari akhir ke awal
91 public void tampilMundur_1013() {
92
93     // cek playlist kosong
94     if (head_1013 == null) {
95
96         System.out.println("Playlist kosong.");
97         return;
98     }
99
100    Lagu_2511531013 curr_1013 = head_1013;
101
102    // menuju node terakhir
103    while (curr_1013.next_1013 != null) {
104
105        curr_1013 = curr_1013.next_1013;
106    }
107
108    System.out.println("Playlist Mundur:");
109
110    // traversal mundur
111    while (curr_1013 != null) {
112
113        System.out.println(curr_1013);
114
115        curr_1013 = curr_1013.prev_1013;
116    }
117 }
118
119 // mencari lagu berdasarkan judul
120 public void cariLagu_1013(String judulCari_1013) {
121
122     // cek playlist kosong
123     if (head_1013 == null) {
124
125         System.out.println("Playlist kosong.");
126         return;
127     }
128
129     Lagu_2511531013 curr_1013 = head_1013;
130
131     boolean ditemukan_1013 = false;
132
133     // traversal pencarian lagu
134     while (curr_1013 != null) {
135
136         // pencarian tidak membedakan huruf besar kecil
137         if (curr_1013.getJudul_1013()
138             .equalsIgnoreCase(judulCari_1013)) {
139
140             System.out.println(
141                 "Lagu ditemukan:");
142
143             System.out.println(curr_1013);
144
145             ditemukan_1013 = true;
146
147             break;
148         }
149
150         curr_1013 = curr_1013.next_1013;
151     }
152
153     // jika lagu tidak ditemukan
154     if (!ditemukan_1013) {
155
156         System.out.println(
157             "Lagu tidak ditemukan.");
158     }
159 }
160
161 // method menampilkan menu
162 public static void tampilMenu_1013() {
163
164     System.out.println(
165         "\n=== Playlist Musik NIM: 2511531013 ===");
166
167     System.out.println("1. Tambah Lagu");
168     System.out.println("2. Hapus Lagu Pertama");
169     System.out.println("3. Lihat Playlist");
170     System.out.println("4. Lihat Playlist (Maju)");
171     System.out.println("5. Cari Lagu");
172     System.out.println("6. Keluar");
173
174     System.out.print("Pilihan : ");
175 }
176
177 public static void main(String[] args) {
178
179     Scanner input_1013 =
180         new Scanner(System.in);
181
182     Musik_2511531013 playlist_1013 =
183         new Musik_2511531013();
184
185     int pilihan_1013;
186
187     do {
188
189         tampilMenu_1013();
190
191         pilihan_1013 =
192             input_1013.nextInt();
193     }

```

```

194         input_1013.nextLine();
195
19600     switch (pilihan_1013) {
197
19800         case 1:
199
200             System.out.print(
201                 "Judul lagu : ");
202
203             String judul_1013 =
204                 input_1013.nextLine();
205
206             System.out.print(
207                 "Penyanyi : ");
208
209             String penyanyi_1013 =
210                 input_1013.nextLine();
211
212             playlist_1013.tambahLagu_1013(
213                 judul_1013,
214                 penyanyi_1013);
215
216             break;
217
21800         case 2:
219
220             playlist_1013
221                 .hapusLaguawal_1013();
222
223             break;
224
22500         case 3:
226
227             playlist_1013
228                 .tampilHajj_1013();
229
230             break;
231
23200         case 4:
233
234             playlist_1013
235                 .tampilPondor_1013();
236
237             break;
238
23900         case 5:
240
241             System.out.print(
242                 "Masukkan judul lagu : ");
243
244             String cari_1013 =
245                 input_1013.nextLine();
246
247             playlist_1013
248                 .cariLagu_1013(cari_1013);
249
250             break;
251
25200         case 6:
253
254             System.out.println(
255                 "Keluar dari program.");
256
257             break;
258
25900         default:
260
261             System.out.println(
262                 "Pilihan tidak valid.");
263
264             } while (pilihan_1013 != 6);
265
266             input_1013.close();
267
268         }
269     }

```

Program Musik_2511531013 digunakan untuk mengelola playlist lagu menggunakan struktur data Double Linked List. Pada program ini terdapat pointer head_1013 yang digunakan untuk menunjuk lagu pertama dalam playlist.

Method tambahLagu_1013() digunakan untuk menambahkan lagu baru di bagian akhir playlist. Program akan membuat node lagu baru menggunakan class Lagu_2511531013. Jika playlist masih kosong, maka node baru langsung menjadi head_1013. Namun jika playlist sudah memiliki data, program akan melakukan traversal sampai node terakhir, kemudian node baru disambungkan di bagian akhir playlist menggunakan pointer next_1013 dan prev_1013.

Method `hapusLaguAwal_1013()` digunakan untuk menghapus lagu pertama pada playlist. Jika playlist kosong, maka program akan menampilkan pesan bahwa playlist kosong. Jika terdapat data, maka pointer `head_1013` akan digeser ke node berikutnya sehingga lagu pertama terhapus dari playlist. Setelah itu, pointer `prev_1013` pada head baru diatur menjadi null.

Method `tampilMaju_1013()` digunakan untuk menampilkan seluruh lagu dari awal ke akhir playlist. Program melakukan traversal mulai dari `head_1013` menggunakan pointer `next_1013` sampai mencapai node terakhir.

Method `tampilMundur_1013()` digunakan untuk menampilkan seluruh lagu dari akhir ke awal playlist. Program terlebih dahulu mencari node terakhir, kemudian melakukan traversal mundur menggunakan pointer `prev_1013` sampai kembali ke node pertama.

Method `cariLagu_1013()` digunakan untuk mencari lagu berdasarkan judul. Pencarian dilakukan dengan menelusuri playlist dari awal hingga akhir menggunakan traversal. Program menggunakan `equalsIgnoreCase()` sehingga pencarian tidak membedakan huruf besar dan huruf kecil.

Method `tampilMenu_1013()` digunakan untuk menampilkan daftar menu program seperti tambah lagu, hapus lagu pertama, melihat playlist secara maju dan mundur, mencari lagu, serta keluar dari program.

Pada method `main`, program menggunakan Scanner untuk menerima input dari pengguna. Program dijalankan menggunakan perulangan `do-while` agar menu terus tampil sampai pengguna memilih keluar. Setiap

pilihan menu diproses menggunakan switch-case sesuai operasi yang dipilih pengguna.

3. Output

```
=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul Lagu : haha
Penyanyi : hihi
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul Lagu : huhu
Penyanyi : hehe
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3
Playlist Maju:
Judul : haha, Penyanyi : hihi
Judul : huhu, Penyanyi : hehe

=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 4
Playlist Mundur:
Judul : huhu, Penyanyi : hehe
Judul : haha, Penyanyi : hihi

=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 5
Masukkan judul lagu : huhu
Lagu ditemukan:
Judul : huhu, Penyanyi : hehe

=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu dihapus : Judul : haha, Penyanyi : hihi

=== Playlist Musik NIM: 2511531013 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
```